WINTER CAMP · WEEK 1 · CLASS 1

Async JavaScript

callbacks → promises → async/await

Today's goal: build a terminal app that reads and writes files without blocking.

TODAY

By the end of today, you can...

1. Explain **why** JavaScript needs asynchronous code

TODAY

By the end of today, you can...

1. Explain **why** JavaScript needs asynchronous code
2. Read and write **callbacks, promises and async/await**, and convert between them

TODAY

By the end of today, you can...

1. Explain **why** JavaScript needs asynchronous code
2. Read and write **callbacks, promises and async/await**, and convert between them
3. Build a **Node.js command-line app** that saves data to a file (your first assignment)

PREREQUISITES

You should already know basic JS syntax

Variables

```
const name = "Asha";
```

Objects

```
{ id: 1, title: "Buy milk" }
```

Arrays

```
tasks.push(task); tasks.length
```

if / else

```
if (done) { … } else { … }
```

Loops

```
for (const t of tasks) { … }
```

Functions

```
const add = (a, b) ⇒ a + b;
```

Classes

```
class Task { constructor(title) { this.title = title; } }  // new Task("Buy milk")
```

Rusty on any of these? Revise them before Thursday. The assignment uses **all of them.**

TODAY

The plan

01 JavaScript outside the browser

02 Why async? CPUs and threads

03 Callbacks

04 Promises

05 async / await

06 Toolkit: argv, JSON, fs/promises

07 Your assignment

08 Bonus: Python and Rust

01

JavaScript outside the browser

Node.js, running files, built-in modules, and Bun

NODE.JS

Same language, new home

In the browser

- document, window, the DOM
- Clicks, forms, page updates
- Can’t touch files on your computer

NODE.JS

Same language, new home

In the browser

- document, window, the DOM
- Clicks, forms, page updates
- Can’t touch files on your computer

In Node.js

- No DOM, no window
- Read and write files (fs)
- Runs from the terminal (process)

NODE.JS

Same language, new home

In the browser

- document, window, the DOM
- Clicks, forms, page updates
- Can't touch files on your computer

In Node.js

- No DOM, no window
- Read and write files (fs)
- Runs from the terminal (process)

Node.js = Chrome's JavaScript engine (V8) + access to your computer: files, network, terminal.

NODE.JS

Running a file

index.js

```
console.log("Hello from Node!");
```

NODE.JS

Running a file

index.js

```
console.log("Hello from Node!");
```

Terminal

```
$ node index.js
Hello from Node!
```

NODE.JS

Built-in modules

```
// files, callback style
const fs = require("fs");

// files, promise style (we'll use this one)
const fsp = require("fs/promises");
```

NODE.JS

Built-in modules

```
// files, callback style
const fs = require("fs");

// files, promise style (we'll use this one)
const fsp = require("fs/promises");
```

> These ship with Node. No **npm install** needed.

RUNTIMES

Node isn’t the only runtime

Node.js

- Engine: V8 (from Chrome)
- Around since 2009
- The industry default

RUNTIMES

Node isn't the only runtime

Node.js

- Engine: V8 (from Chrome)
- Around since 2009
- The industry default

Bun

- Engine: JavaScriptCore (from Safari)
- Written in Zig, built for speed
- Package manager, test runner and bundler built in

RUNTIMES

Same file, two runtimes

index.js

```
const fs = require("fs/promises");

async function main() {
  const text = await fs.readFile("a.txt",
"utf-8");
  console.log(text);
}
main();
```

Terminal

```
$ node index.js
hello
$ bun index.js
hello
```

RUNTIMES

Same file, two runtimes

index.js

```
const fs = require("fs/promises");

async function main() {
  const text = await fs.readFile("a.txt",
"utf-8");
  console.log(text);
}
main();
```

Terminal

```
$ node index.js
hello
$ bun index.js
hello
```

Runtime = a JS engine + APIs for files, network and timers. The language and your async code stay the same.

RUNTIMES

Bun: batteries included

`bun index.ts`	Runs TypeScript directly, no setup
`bun install`	Installs packages, like npm install
`bun test`	Built-in test runner
`bun build`	Bundles your code for shipping

For Assignment 01, use **Node.** Trying it with Bun afterwards is a fun bonus.

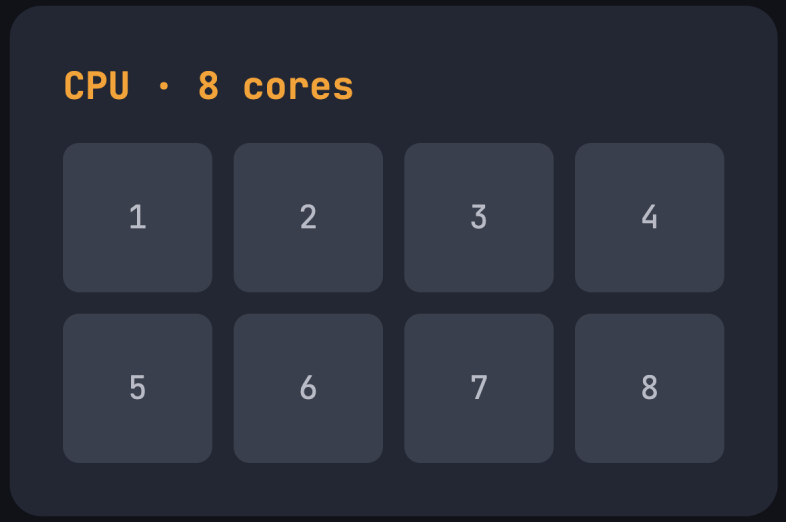
02

Why async?

CPUs, threads, blocking code and the event loop

WHY ASYNC?

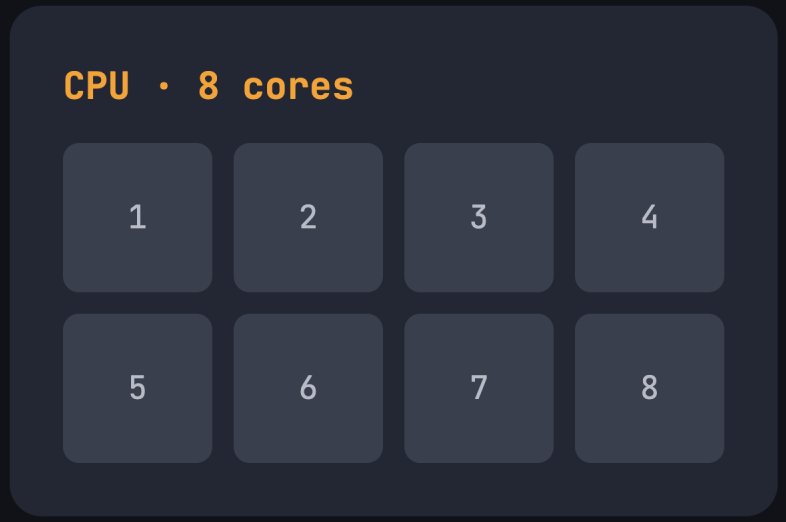
Inside your machine



> A **CPU** has several **cores**. Each core runs one thing at a time, so 8 cores can do 8 things at *truly* the same instant.

WHY ASYNC?

Inside your machine

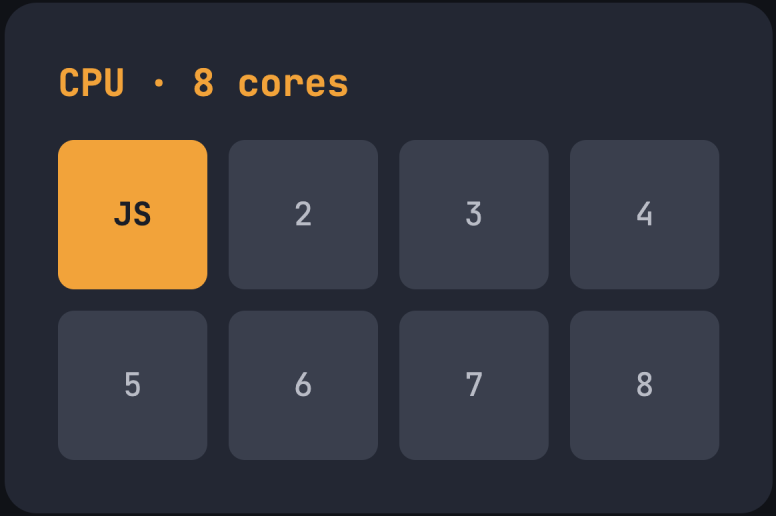


A **CPU** has several **cores**. Each core runs one thing at a time, so 8 cores can do 8 things at *truly* the same instant.

A **thread** is one line of work that a core executes. A program can use one thread, or many.

WHY ASYNC?

Inside your machine



A **CPU** has several **cores**. Each core runs one thing at a time, so 8 cores can do 8 things at *truly* the same instant.

A **thread** is one line of work that a core executes. A program can use one thread, or many.

JavaScript runs **your code on one thread**. One call stack, one line at a time. That's what "single-threaded" means.

WHY ASYNC?

Inside your machine

A **CPU** has several **cores**. Each core runs one thing at a time, so 8 cores can do 8 things at *truly* the same instant.

A **thread** is one line of work that a core executes. A program can use one thread, or many.

JavaScript runs **your code on one thread**. One call stack, one line at a time. That's what "single-threaded" means.

So who reads the file? Node hands slow work to the **OS and its own background threads**, and gets a message when it's done. **That's where async comes in.**

WHY ASYNC?

Parallel vs concurrent

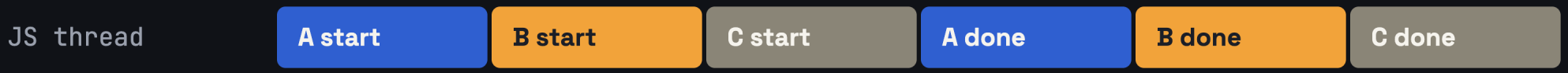
Parallel: many workers, at the same instant

WHY ASYNC?

Parallel vs concurrent

Parallel: many workers, at the same instant

Concurrent: one worker, juggling



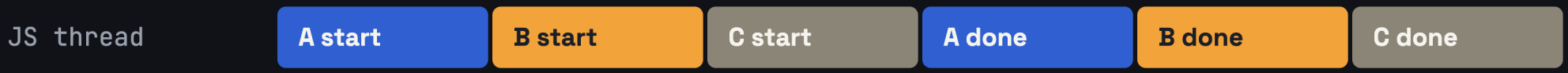
Between “start” and “done”, each task is waiting on a file or the network, not using the thread.

WHY ASYNC?

Parallel vs concurrent

Parallel: many workers, at the same instant

Concurrent: one worker, juggling



Between “start” and “done”, each task is waiting on a file or the network, not using the thread.

> JavaScript is **concurrent, not parallel.** Async doesn’t add workers. It makes sure the one worker **never stands around waiting.**

WHY ASYNC?

Two kinds of waiter

The blocking waiter

1. Takes table 1's order
2. Stands at the kitchen door until it's cooked
3. Serves table 1
4. Only then walks to table 2

WHY ASYNC?

Two kinds of waiter

The blocking waiter

1. Takes table 1's order
2. Stands at the kitchen door until it's cooked
3. Serves table 1
4. Only then walks to table 2

The async waiter

1. Hands table 1's order to the kitchen
2. Takes orders from tables 2 and 3
3. Kitchen rings a bell: serves table 1
4. Never stands around doing nothing

WHY ASYNC?

Two kinds of waiter

The blocking waiter

1. Takes table 1's order
2. Stands at the kitchen door until it's cooked
3. Serves table 1
4. Only then walks to table 2

The async waiter

1. Hands table 1's order to the kitchen
2. Takes orders from tables 2 and 3
3. Kitchen rings a bell: serves table 1
4. Never stands around doing nothing

JavaScript is the waiter, and **there is only one.** Async code keeps it serving while slow work happens somewhere else.

WHY ASYNC?

The blocking waiter, in code

```
const fs = require("fs");

console.log("1. Start");
const data = fs.readFileSync("huge-file.txt", "utf-8");
console.log("2. Finished reading");
console.log("3. Now I can do other work");
```

WHY ASYNC?

The blocking waiter, in code

```
const fs = require("fs");

console.log("1. Start");
const data = fs.readFileSync("huge-file.txt", "utf-8");
console.log("2. Finished reading");
console.log("3. Now I can do other work");
```

While **readFileSync** runs, nothing else can run. On a server, every other user waits too.

WHY ASYNC?

What counts as “slow”?

Files

Reading from or writing to disk

Network

Calling an API, talking to a database

Timers

Waiting on purpose with setTimeout

WHY ASYNC?

What counts as “slow”?

Files

Reading from or writing to disk

Network

Calling an API, talking to a database

Timers

Waiting on purpose with setTimeout

Rule of thumb: if it waits on the **outside world**, do it async.

WHY ASYNC?

Predict the output

index.js

```
console.log("A");

setTimeout(() => {
  console.log("B");
}, 0);

console.log("C");
```

Output

```
$ node index.js
?
```

WHY ASYNC?

Predict the output

index.js

```
console.log("A");

setTimeout(() => {
  console.log("B");
}, 0);

console.log("C");
```

Output

```
$ node index.js
A
C
B
```

WHY ASYNC?

Predict the output

index.js

```
console.log("A");

setTimeout(() => {
  console.log("B");
}, 0);

console.log("C");
```

Output

```
$ node index.js
A
C
B
```

B waits even with 0 ms. What decides the order? **The event loop.**

WHY ASYNC?

The event loop, step by step

```
console.log("A");
setTimeout(() => {
  console.log("B");
}, 0);
console.log("C");
```

Output:

1 · Call stack

Running right now

2 · Node APIs

Timers, files, network: waiting

3 · Callback queue

Done waiting, ready to run

WHY ASYNC?

The event loop, step by step

```
console.log("A");
setTimeout(() ⇒ {
  console.log("B");
}, 0);
console.log("C");
```

```
Output: A
```

1 · Call stack

Running right now

```
console.log("A")
```

2 · Node APIs

Timers, files, network: waiting

3 · Callback queue

Done waiting, ready to run

WHY ASYNC?

The event loop, step by step

```
console.log("A");
setTimeout(() => {
  console.log("B");
}, 0);
console.log("C");
```

Output: A

1 · Call stack

Running right now

`setTimeout(…)`

2 · Node APIs

Timers, files, network: waiting

`timer: 0 ms`

3 · Callback queue

Done waiting, ready to run

WHY ASYNC?

The event loop, step by step

```
console.log("A");
setTimeout(() ⇒ {
  console.log("B");
}, 0);
console.log("C");
```

Output: A · C

1 · Call stack

Running right now

`console.log("C")`

2 · Node APIs

Timers, files, network: waiting

3 · Callback queue

Done waiting, ready to run

`() ⇒ log("B")`

WHY ASYNC?

The event loop, step by step

```
console.log("A");
setTimeout(() ⇒ {
  console.log("B");
}, 0);
console.log("C");
```

Output: A · C

1 · Call stack

Running right now

(empty)

2 · Node APIs

Timers, files, network: waiting

3 · Callback queue

Done waiting, ready to run

`() ⇒ log("B")`

The **event loop**: when the stack is empty, move the next callback from the queue onto the stack.

WHY ASYNC?

The event loop, step by step

```
console.log("A");
setTimeout(() ⇒ {
  console.log("B");
}, 0);
console.log("C");
```

Output: A · C · B

1 · Call stack

Running right now

`() ⇒ log("B")`

2 · Node APIs

Timers, files, network: waiting

3 · Callback queue

Done waiting, ready to run

The **event loop**: when the stack is empty, move the next callback from the queue onto the stack.

03

Callbacks

“Call me back when you’re done”

CALLBACKS

You've already used callbacks

```
button.addEventListener("click", () ⇒ {
  console.log("Clicked!");
});
```

CALLBACKS

You've already used callbacks

```
button.addEventListener("click", () ⇒ {
  console.log("Clicked!");
});
```

A **callback** is a function you hand to someone else, to be called later, when something happens.

CALLBACKS

Reading a file, the async way

index.js

```
const fs = require("fs");

console.log("1. Start");

fs.readFile("notes.txt", "utf-8", (err, data) => {
  console.log("3. File:", data);
});

console.log("2. Not blocked!");
```

CALLBACKS

Reading a file, the async way

index.js

```
const fs = require("fs");

console.log("1. Start");

fs.readFile("notes.txt", "utf-8", (err, data) => {
  console.log("3. File:", data);
});

console.log("2. Not blocked!");
```

Terminal

```
$ node index.js
1. Start
2. Not blocked!
3. File: hello
```

CALLBACKS

Error-first callbacks

```
fs.readFile("missing.txt", "utf-8", (err, data) ⇒ {
  if (err) {
    console.log("Oops:", err.message);
    return;
  }
  console.log(data);
});
```

CALLBACKS

Error-first callbacks

```
fs.readFile("missing.txt", "utf-8", (err, data) ⇒ {
  if (err) {
    console.log("Oops:", err.message);
    return;
  }
  console.log(data);
});
```

err

1st argument: the error, or null if it worked

data

2nd argument: the result, if it worked

YOUR TURN · 10 MINUTES

Write it with callbacks

1. Read **a.txt** with fs.readFile
2. Convert the text to **UPPER CASE**
3. Write it to **b.txt** with fs.writeFile
4. Check **err** at every step

Callbacks only. No promises, no async/await yet.

YOUR TURN · 10 MINUTES

Write it with callbacks

1. Read **a.txt** with fs.readFile
2. Convert the text to **UPPER CASE**
3. Write it to **b.txt** with fs.writeFile
4. Check **err** at every step

Callbacks only. No promises, no async/await yet.

When it works

```
$ cat a.txt
hello world
$ node index.js
Done!
$ cat b.txt
HELLO WORLD
```

Hint: fs.writeFile(path, data, (err) => …). Where does the write have to go?

CALLBACKS

Callback hell

Task: read a.txt → write it in CAPS to b.txt → read b.txt back

```
fs.readFile("a.txt", "utf-8", (err, a) ⇒ {
  if (err) return console.log(err);
});
```

CALLBACKS

Callback hell

Task: read a.txt → write it in CAPS to b.txt → read b.txt back

```
fs.readFile("a.txt", "utf-8", (err, a) ⇒ {
  if (err) return console.log(err);
  fs.writeFile("b.txt", a.toUpperCase(), (err) ⇒ {
    if (err) return console.log(err);
  });
});
```

CALLBACKS

Callback hell

Task: read a.txt → write it in CAPS to b.txt → read b.txt back

```
fs.readFile("a.txt", "utf-8", (err, a) ⇒ {
  if (err) return console.log(err);
  fs.writeFile("b.txt", a.toUpperCase(), (err) ⇒ {
    if (err) return console.log(err);
    fs.readFile("b.txt", "utf-8", (err, b) ⇒ {
      if (err) return console.log(err);
      console.log(b);
    });
  });
});
```

CALLBACKS

Callback hell

Task: read a.txt → write it in CAPS to b.txt → read b.txt back

```
fs.readFile("a.txt", "utf-8", (err, a) => {
  if (err) return console.log(err);
  fs.writeFile("b.txt", a.toUpperCase(), (err) => {
    if (err) return console.log(err);
    fs.readFile("b.txt", "utf-8", (err, b) => {
      if (err) return console.log(err);
      console.log(b);
    });
  });
});
```

The **pyramid of doom**: every step nests deeper, and error handling repeats at every level.

04

Promises

Flattening the pyramid

PROMISES

A promise is a food-court buzzer

You order and get a **buzzer**. No food yet. The buzzer is the promise, and right now it's **pending**.

PROMISES

A promise is a food-court buzzer

You order and get a **buzzer**. No food yet. The buzzer is the promise, and right now it's **pending**.

It buzzes and your food is ready: **fulfilled**, with a value (your food).

PROMISES

A promise is a food-court buzzer

You order and get a **buzzer**. No food yet. The buzzer is the promise, and right now it's **pending**.

It buzzes and your food is ready: **fulfilled**, with a value (your food).

Or: "Sorry, we're out of paneer." **Rejected**, with a reason (an error).

PROMISES

Three states

`pending`

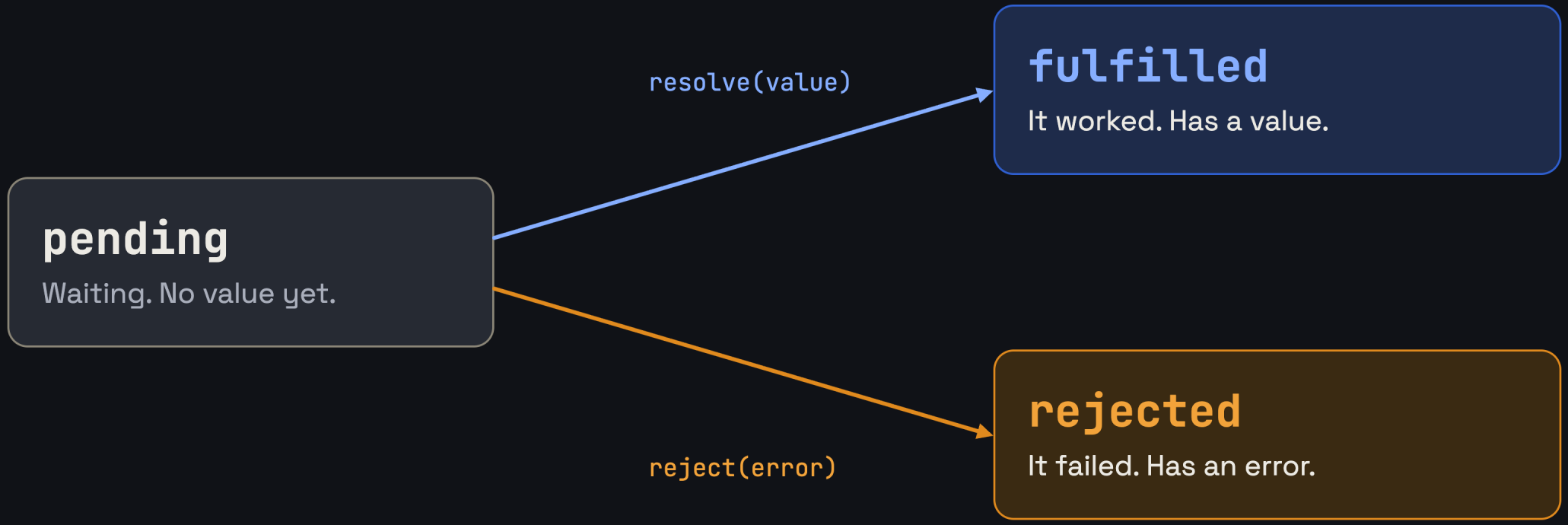
Waiting. No value yet.

PROMISES

Three states

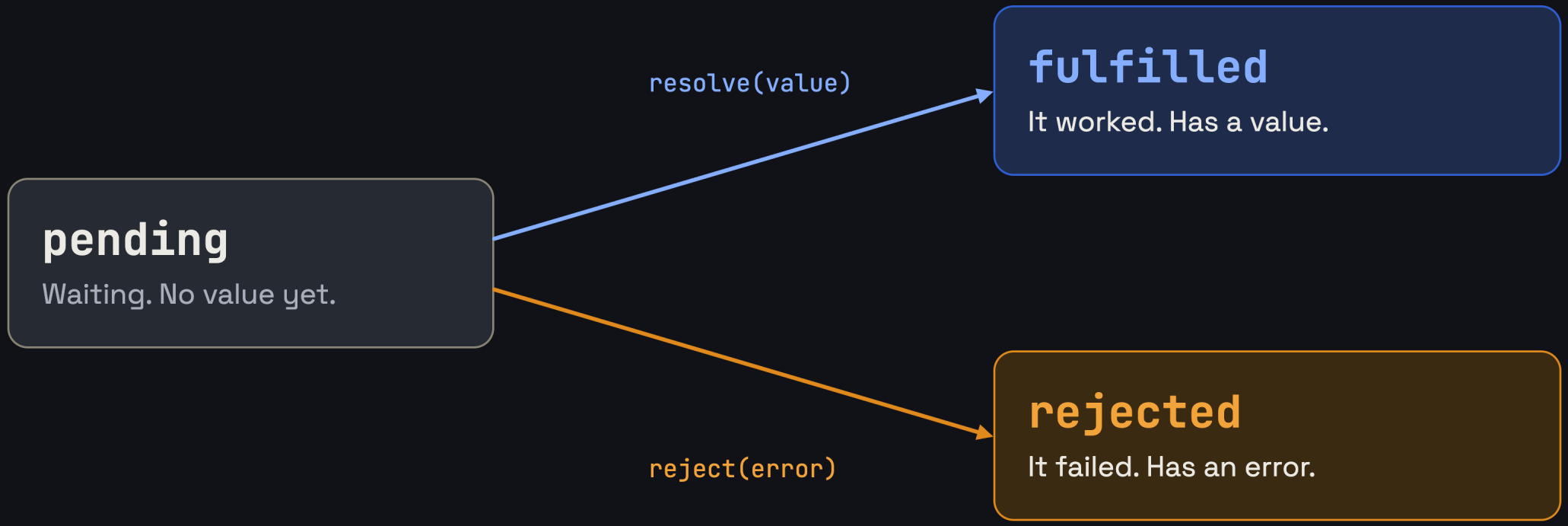
PROMISES

Three states



PROMISES

Three states



Once a promise **settles** (fulfilled or rejected), it never changes again.

PROMISES

Using a promise

fs/promises has the same functions as fs, but they **return a promise** instead of taking a callback.

```
const fs = require("fs/promises");

fs.readFile("notes.txt", "utf-8")
```

PROMISES

Using a promise

fs/promises has the same functions as fs, but they **return a promise** instead of taking a callback.

```
const fs = require("fs/promises");

fs.readFile("notes.txt", "utf-8")
  .then((data) ⇒ console.log(data))  // fulfilled
```

PROMISES

Using a promise

fs/promises has the same functions as fs, but they **return a promise** instead of taking a callback.

```
const fs = require("fs/promises");

fs.readFile("notes.txt", "utf-8")
  .then((data) ⇒ console.log(data))  // fulfilled
  .catch((err) ⇒ console.log("Oops:", err.message))  // rejected
```

PROMISES

Using a promise

fs/promises has the same functions as fs, but they **return a promise** instead of taking a callback.

```
const fs = require("fs/promises");

fs.readFile("notes.txt", "utf-8")
  .then((data) ⇒ console.log(data))  // fulfilled
  .catch((err) ⇒ console.log("Oops:", err.message))  // rejected
  .finally(() ⇒ console.log("Done either way"));  // always
```

PROMISES

Chaining flattens the pyramid

Same task: read a.txt → write it in CAPS to b.txt → read b.txt back

```
// fs = require("fs/promises")
fs.readFile("a.txt", "utf-8")
  .then((a) ⇒ fs.writeFile("b.txt", a.toUpperCase()))
```

PROMISES

Chaining flattens the pyramid

Same task: read a.txt → write it in CAPS to b.txt → read b.txt back

```
// fs = require("fs/promises")
fs.readFile("a.txt", "utf-8")
  .then((a) ⇒ fs.writeFile("b.txt", a.toUpperCase()))
  .then(() ⇒ fs.readFile("b.txt", "utf-8"))
  .then((b) ⇒ console.log(b))
```

PROMISES

Chaining flattens the pyramid

Same task: read a.txt → write it in CAPS to b.txt → read b.txt back

```
// fs = require("fs/promises")
fs.readFile("a.txt", "utf-8")
  .then((a) ⇒ fs.writeFile("b.txt", a.toUpperCase()))
  .then(() ⇒ fs.readFile("b.txt", "utf-8"))
  .then((b) ⇒ console.log(b))
  .catch((err) ⇒ console.log(err));
```

Return a promise from **.then** and the next .then waits for it. **One .catch** handles an error from any step.

PROMISES

Making your own promise

```
function sleep(ms) {
  return new Promise((resolve, reject) => {
    setTimeout(() => resolve("done"), ms);
  });
}
```

PROMISES

Making your own promise

```
function sleep(ms) {
  return new Promise((resolve, reject) => {
    setTimeout(() => resolve("done"), ms);
  });
}

sleep(2000).then((msg) => console.log(msg));  // "done", 2s later
```

PROMISES

Making your own promise

```
function sleep(ms) {
  return new Promise((resolve, reject) => {
    setTimeout(() => resolve("done"), ms);
  });
}

sleep(2000).then((msg) => console.log(msg));  // "done", 2s later
```

Call **resolve(value)** to fulfil it, or **reject(error)** to reject it. Most days you'll *use* promises, not make them.

PROMISES

Predict the output, round 2

index.js

```
console.log("1");

setTimeout(() ⇒ console.log("2"), 0);

Promise.resolve().then(() ⇒ console.log("3"));

console.log("4");
```

Output

```
$ node index.js
?
```

PROMISES

Predict the output, round 2

index.js

```
console.log("1");

setTimeout(() ⇒ console.log("2"), 0);

Promise.resolve().then(() ⇒ console.log("3"));

console.log("4");
```

Output

```
$ node index.js
1
4
3
2
```

PROMISES

Predict the output, round 2

index.js

```
console.log("1");

setTimeout(() ⇒ console.log("2"), 0);

Promise.resolve().then(() ⇒ console.log("3"));

console.log("4");
```

Output

```
$ node index.js
1
4
3
2
```

Promise callbacks have their own **VIP queue** (the microtask queue). It is emptied before any timer callback runs.

05

async / await

Promises that read like normal code

ASYNC / AWAIT

Two new keywords

```
const fs = require("fs/promises");

async function main() {

}

main();
```

> **async:** this function is allowed to pause with await.

ASYNC / AWAIT

Two new keywords

```
const fs = require("fs/promises");

async function main() {
  const data = await fs.readFile("notes.txt", "utf-8");
}

main();
```

async: this function is allowed to pause with await.

await: pause here until the promise settles, then give me its value.

ASYNC / AWAIT

Two new keywords

```
const fs = require("fs/promises");

async function main() {
  const data = await fs.readFile("notes.txt", "utf-8");
  console.log(data);
}

main();
```

async: this function is allowed to pause with await.

await: pause here until the promise settles, then give me its value.

Only **main** pauses. The thread stays free for other work.

ASYNC / AWAIT

The pyramid, one last time

Same task: read a.txt → write it in CAPS to b.txt → read b.txt back

```
async function main() {
  const a = await fs.readFile("a.txt", "utf-8");
}
```

ASYNC / AWAIT

The pyramid, one last time

Same task: read a.txt → write it in CAPS to b.txt → read b.txt back

```
async function main() {
  const a = await fs.readFile("a.txt", "utf-8");
  await fs.writeFile("b.txt", a.toUpperCase());
  const b = await fs.readFile("b.txt", "utf-8");
}
```

ASYNC / AWAIT

The pyramid, one last time

Same task: read a.txt → write it in CAPS to b.txt → read b.txt back

```
async function main() {
  const a = await fs.readFile("a.txt", "utf-8");
  await fs.writeFile("b.txt", a.toUpperCase());
  const b = await fs.readFile("b.txt", "utf-8");
  console.log(b);
}
```

Callbacks → promises → async/await: **same steps, same speed**, just easier to read.

ASYNC / AWAIT

Handling errors: try / catch

If the promise rejects, **await throws**, just like a normal error.

```
async function main() {
  const data = await fs.readFile("missing.txt", "utf-8");
  console.log(data);
}
```

ASYNC / AWAIT

Handling errors: try / catch

If the promise rejects, **await throws**, just like a normal error.

```
async function main() {
  try {
    const data = await fs.readFile("missing.txt", "utf-8");
    console.log(data);
  } catch (err) {
    console.log("Could not read:", err.message);
  }
}
```

ASYNC / AWAIT

Handling errors: try / catch

If the promise rejects, **await throws**, just like a normal error.

```
async function main() {
  try {
    const data = await fs.readFile("missing.txt", "utf-8");
    console.log(data);
  } catch (err) {
    console.log("Could not read:", err.message);
  }
}
```

```
$ node index.js
Could not read: ENOENT: no such file or directory, open 'missing.txt'
```

ASYNC / AWAIT

Three classic mistakes

1. Forgetting await

You get the buzzer, not the food.

```
const data = fs.readFile(f, "utf-8");
console.log(data);  // Promise { <pending> }
```

ASYNC / AWAIT

Three classic mistakes

1. Forgetting await

You get the buzzer, not the food.

```
const data = fs.readFile(f, "utf-8");
console.log(data);  // Promise { <pending> }
```

2. await outside async

await only works inside an async function.

```
function main() { await fs.readFile(f); }
// SyntaxError: await is only valid in async functions
```

ASYNC / AWAIT

Three classic mistakes

1. Forgetting await

You get the buzzer, not the food.

```
const data = fs.readFile(f, "utf-8");
console.log(data);  // Promise { <pending> }
```

2. await outside async

await only works inside an async function.

```
function main() { await fs.readFile(f); }
// SyntaxError: await is only valid in async functions
```

3. Expecting a plain value

An async function always returns a promise.

```
async function getFive() { return 5; }
console.log(getFive());  // Promise { 5 }
```

06

Your toolkit

process.argv, JSON, fs/promises, and a live demo

TOOLKIT

Reading terminal input: process.argv

index.js

```
console.log(process.argv);
```

Terminal

```
$ node index.js add "Buy milk"
```

TOOLKIT

Reading terminal input: process.argv

index.js

```
console.log(process.argv);
```

Terminal

```
$ node index.js add "Buy milk"
[
  '/usr/local/bin/node',
  '/Users/you/task-manager/index.js',
  'add',
  'Buy milk'
]
```

argv[0] is node and argv[1] is your file. **Your input starts at index 2.**

TOOLKIT

Grab just what you need

index.js

```
const [, , command, ...args] = process.argv;

console.log(command);
console.log(args);
```

TOOLKIT

Grab just what you need

index.js

```
const [, , command, ...args] = process.argv;

console.log(command);
console.log(args);
```

Terminal

```
$ node index.js update 1 "New title"
update
[ '1', 'New title' ]
```

TOOLKIT

Grab just what you need

index.js

```
const [, , command, ...args] = process.argv;

console.log(command);
console.log(args);
```

Terminal

```
$ node index.js update 1 "New title"
update
[ '1', 'New title' ]
```

Everything in argv is a **string**. Need a number for an ID? **Number(args[0])**

TOOLKIT

JSON ↔ JavaScript

```
const text = '[{"id":1,"title":"Buy milk"}]';

const tasks = JSON.parse(text);  // string → array
```

TOOLKIT

JSON ↔ JavaScript

```
const text = '[{"id":1,"title":"Buy milk"}]';

const tasks = JSON.parse(text);  // string → array
tasks.push({ id: 2, title: "Call mom" });

const out = JSON.stringify(tasks, null, 2);  // array → string
```

TOOLKIT

JSON ↔ JavaScript

```
const text = '[{"id":1,"title":"Buy milk"}]';

const tasks = JSON.parse(text);  // string → array
tasks.push({ id: 2, title: "Call mom" });

const out = JSON.stringify(tasks, null, 2);  // array → string
```

Files only hold text. **JSON.parse** after you read, **JSON.stringify** before you write. (null, 2) pretty-prints with 2 spaces.

TOOLKIT

When the JSON is broken

Without try / catch

```
const tasks = JSON.parse("[{ oops");
```

```
SyntaxError: Expected property name or '}'
in JSON at position 3
(program crashes)
```

TOOLKIT

When the JSON is broken

Without try / catch

```
const tasks = JSON.parse("[{ oops");
```

```
SyntaxError: Expected property name or '}'
in JSON at position 3
(program crashes)
```

With try / catch

```
try {
  const tasks = JSON.parse("[{ oops");
} catch (err) {
  console.log("Bad JSON:", err.message);
}
```

```
Bad JSON: Expected property name or '}' in
JSON at position 3
```

TOOLKIT

fs/promises: all you need

```
await fs.readFile("tasks.json", "utf-8")
```

Resolves to the file's contents as a string. Rejects if the file can't be read.

TOOLKIT

fs/promises: all you need

```
await fs.readFile("tasks.json", "utf-8")
```

Resolves to the file's contents as a string. Rejects if the file can't be read.

```
await fs.writeFile("tasks.json", text)
```

Replaces the whole file with text. Creates the file if it doesn't exist.

TOOLKIT

fs/promises: all you need

```
await fs.readFile("tasks.json", "utf-8")
```

Resolves to the file's contents as a string. Rejects if the file can't be read.

```
await fs.writeFile("tasks.json", text)
```

Replaces the whole file with text. Creates the file if it doesn't exist.

```
err.code === "ENOENT"
```

Tells you readFile failed because the file doesn't exist yet. You decide what that means for your app.

LIVE CODING

notes.js: putting it together

```
const fs = require("fs/promises");

async function main() {
  const [, , text] = process.argv;
}

main();
```

LIVE CODING

notes.js: putting it together

```
const fs = require("fs/promises");

async function main() {
  const [, , text] = process.argv;

  const raw = await fs.readFile("notes.json", "utf-8");
  const notes = JSON.parse(raw);
}

main();
```

LIVE CODING

notes.js: putting it together

```
const fs = require("fs/promises");

async function main() {
  const [, , text] = process.argv;

  const raw = await fs.readFile("notes.json", "utf-8");
  const notes = JSON.parse(raw);

  notes.push({ text, at: new Date().toISOString() });
  await fs.writeFile("notes.json", JSON.stringify(notes, null, 2));
  console.log("Saved! Total notes:", notes.length);
}

main();
```

LIVE CODING

Run it

```
$ echo "[]" > notes.json
$ node notes.js "Learn promises"
Saved! Total notes: 1
$ node notes.js "Learn async/await"
Saved! Total notes: 2
```

LIVE CODING

Run it

```
$ echo "[]" > notes.json
$ node notes.js "Learn promises"
Saved! Total notes: 1
$ node notes.js "Learn async/await"
Saved! Total notes: 2
```

Think about it: what if notes.json **doesn't exist**? What if it holds **broken JSON**? Handling both is part of your assignment.

07

Your assignment

Assignment 01 · due Thursday, 11 AM IST

ASSIGNMENT 01

An async, file-based task manager

tasks.json

```
[
  {
    "id": 1,
    "title": "Complete JS assignment",
    "completed": false,
    "createdAt": "2026-10-20T10:00:00.000Z"
  }
]
```

ASSIGNMENT 01

An async, file-based task manager

tasks.json

```
[
  {
    "id": 1,
    "title": "Complete JS assignment",
    "completed": false,
    "createdAt": "2026-10-20T10:00:00.000Z"
  }
]
```

Commands

node index.js …	What it does
add ”Title”	Create a task
list	Show all tasks
get 1	Show task 1
update 1 ”Title”	Rename task 1
done 1	Mark task 1 completed
delete 1	Remove task 1

ASSIGNMENT 01

Ground rules

Use AI to learn, not to write it. Ask it to explain process.argv, promises or an error. Don't ask it for your CRUD functions.

ASSIGNMENT 01

Ground rules

Use AI to learn, not to write it. Ask it to explain process.argv, promises or an error. Don't ask it for your CRUD functions.

Submit a GitHub repo with index.js, tasks.json and a README: how to run it, every command, example inputs and outputs.

ASSIGNMENT 01

Ground rules

Use AI to learn, not to write it. Ask it to explain process.argv, promises or an error. Don't ask it for your CRUD functions.

Submit a GitHub repo with index.js, tasks.json and a README: how to run it, every command, example inputs and outputs.

Due Thursday, 11 AM IST. You may be asked to explain any line, so only submit code you understand.

08 · Bonus

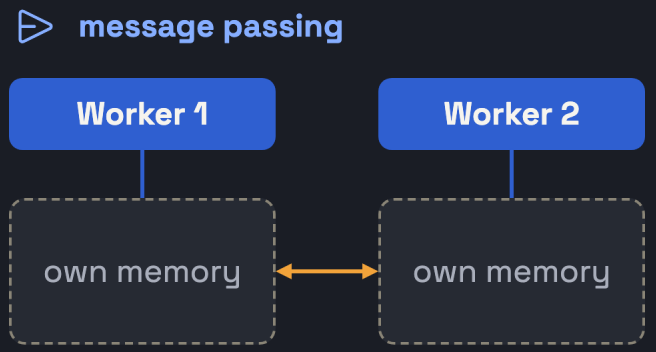
Beyond JavaScript

Python's GIL and asyncio, Rust's async and Tokio

BEYOND JAVASCRIPT

Everyone has threads. The difference is memory.

JS · worker_threads

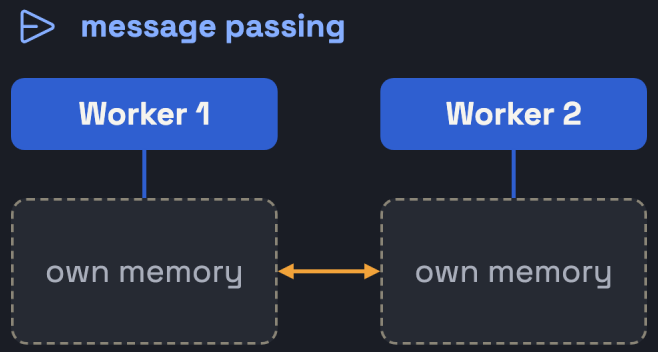


Separate memory, so workers run **truly in parallel**. No lock needed.

BEYOND JAVASCRIPT

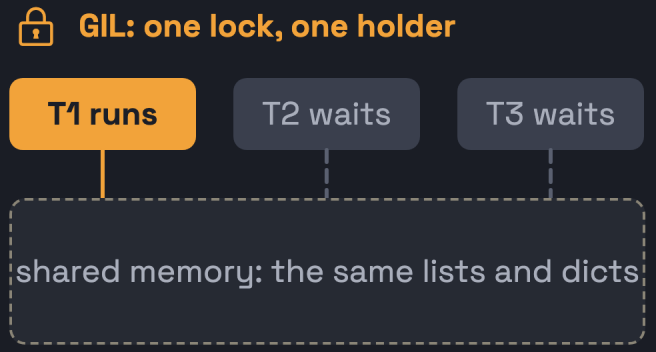
Everyone has threads. The difference is memory.

JS · worker_threads



Separate memory, so workers run **truly in parallel**. No lock needed.

Python · threads



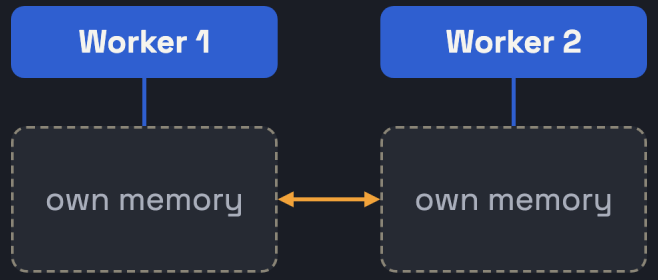
Shared memory, guarded by **one lock**. Only one thread runs Python at a time.

BEYOND JAVASCRIPT

Everyone has threads. The difference is memory.

JS · worker_threads

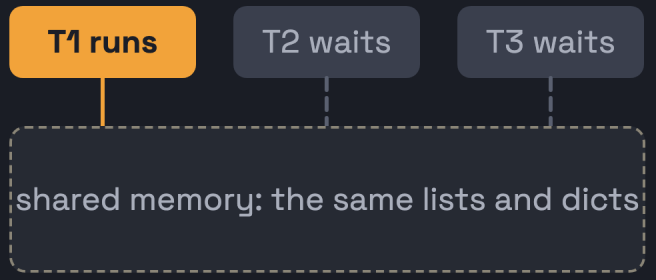
message passing



Separate memory, so workers run **truly in parallel**. No lock needed.

Python · threads

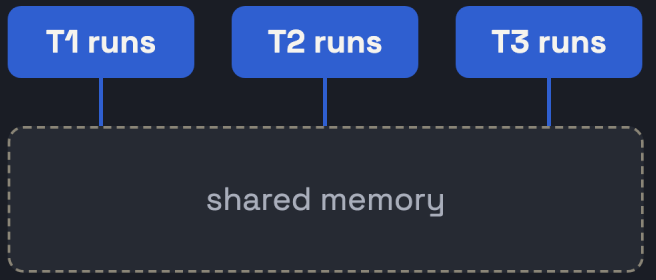
GIL: one lock, one holder



Shared memory, guarded by **one lock**. Only one thread runs Python at a time.

Rust · threads

checked by the compiler



Shared memory **and** all run at once. The compiler rejects unsafe sharing.

BEYOND JAVASCRIPT · CODE

JS in parallel: worker_threads

```
const { Worker, isMainThread, parentPort, workerData }
  = require("worker_threads");

if (isMainThread) {
  for (let i = 1; i ⩽ 3; i++) {
    const w = new Worker(__filename, { workerData: i });
    w.on("message", (msg) ⇒ console.log(msg));
  }
} else {
  // this part runs in its own thread, with its own memory
  let sum = 0;
  for (let n = 0; n < 1e9; n++) sum += n;
  parentPort.postMessage(`worker ${workerData} done`);
}
```

BEYOND JAVASCRIPT · CODE

JS in parallel: worker_threads

```
const { Worker, isMainThread, parentPort, workerData }
  = require("worker_threads");

if (isMainThread) {
  for (let i = 1; i ⩽ 3; i++) {
    const w = new Worker(__filename, { workerData: i });
    w.on("message", (msg) ⇒ console.log(msg));
  }
} else {
  // this part runs in its own thread, with its own memory
  let sum = 0;
  for (let n = 0; n < 1e9; n++) sum += n;
  parentPort.postMessage(`worker ${workerData} done`);
}
```

```
$ node index.js
worker 2 done
worker 1 done
worker 3 done
```

Order changes every run: they **really run at the same time.** Each has its own memory and reports back with **postMessage.**

BEYOND JAVASCRIPT

Python and the GIL

> Python threads are real OS threads that **share memory**. To keep that safe, one lock (the **GIL**) lets only **one thread run Python code at a time**.

BEYOND JAVASCRIPT

Python and the GIL

> Python threads are real OS threads that **share memory**. To keep that safe, one lock (the **GIL**) lets only **one thread run Python code at a time**.

> Threads still help with **waiting**: the GIL is released during file and network I/O.

BEYOND JAVASCRIPT

Python and the GIL

> Python threads are real OS threads that **share memory**. To keep that safe, one lock (the **GIL**) lets only **one thread run Python code at a time**.

> Threads still help with **waiting**: the GIL is released during file and network I/O.

> Heavy CPU work in parallel? Use **multiprocessing**: separate processes, each with its own GIL.

BEYOND JAVASCRIPT

Python and the GIL

> Python threads are real OS threads that **share memory**. To keep that safe, one lock (the **GIL**) lets only **one thread run Python code at a time**.

> Threads still help with **waiting**: the GIL is released during file and network I/O.

> Heavy CPU work in parallel? Use **multiprocessing**: separate processes, each with its own GIL.

main.py · asyncio, the same idea as JS

```python
import asyncio

async def main():
    await asyncio.sleep(1)
    print("done after 1s")

asyncio.run(main())
# ↑ starts an event loop
```

BEYOND JAVASCRIPT · CODE

Python: threads vs processes

threading: takes turns (GIL)

```
import threading
def work(n):
    total = sum(range(10**8))
    print(f"thread {n} done")
ts = [threading.Thread(target=work, args=(i,))
      for i in range(3)]
for t in ts: t.start()
for t in ts: t.join()
```

BEYOND JAVASCRIPT · CODE

Python: threads vs processes

threading: takes turns (GIL)

```
import threading
def work(n):
    total = sum(range(10**8))
    print(f"thread {n} done")
ts = [threading.Thread(target=work, args=(i,))
      for i in range(3)]
for t in ts: t.start()
for t in ts: t.join()
```

multiprocessing: truly parallel

```
from multiprocessing import Process
def work(n):
    total = sum(range(10**8))
    print(f"process {n} done")
if __name__ == "__main__":
    ps = [Process(target=work, args=(i,))
          for i in range(3)]
    for p in ps: p.start()
    for p in ps: p.join()
```

BEYOND JAVASCRIPT · CODE

Python: threads vs processes

threading: takes turns (GIL)

```
import threading
def work(n):
    total = sum(range(10**8))
    print(f"thread {n} done")
ts = [threading.Thread(target=work, args=(i,))
      for i in range(3)]
for t in ts: t.start()
for t in ts: t.join()
```

multiprocessing: truly parallel

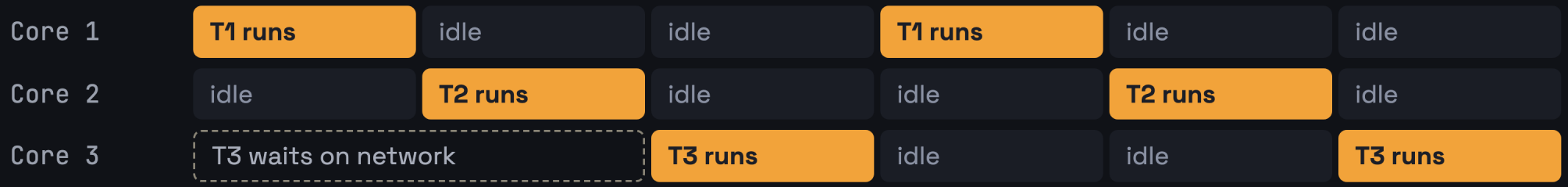
```
from multiprocessing import Process
def work(n):
    total = sum(range(10**8))
    print(f"process {n} done")
if __name__ == "__main__":
    ps = [Process(target=work, args=(i,))
          for i in range(3)]
    for p in ps: p.start()
    for p in ps: p.join()
```

Swap **Thread** for **Process** and CPU work finally spreads across cores, because each process gets **its own GIL.**

BEYOND JAVASCRIPT

What the cores are doing

Python: 3 threads, 3 cores, the GIL



Read it column by column: **never two amber blocks at the same moment.** Waiting on I/O doesn't need the GIL, so it can overlap.

BEYOND JAVASCRIPT

Rust: async with a runtime

> Rust threads are real OS threads with **no GIL**.
> They run **truly in parallel** on every core.

BEYOND JAVASCRIPT

Rust: async with a runtime

> Rust threads are real OS threads with **no GIL**. They run **truly in parallel** on every core.

> Rust has **async/await** too, but **no built-in event loop**. A future does nothing until something runs it.

BEYOND JAVASCRIPT

Rust: async with a runtime

> Rust threads are real OS threads with **no GIL**. They run **truly in parallel** on every core.

> Rust has **async/await** too, but **no built-in event loop**. A future does nothing until something runs it.

> You pick a **runtime**, usually **Tokio**: an event loop plus a pool of threads. Async tasks become concurrent *and* parallel.

BEYOND JAVASCRIPT

Rust: async with a runtime

> Rust threads are real OS threads with **no GIL**. They run **truly in parallel** on every core.

> Rust has **async/await** too, but **no built-in event loop**. A future does nothing until something runs it.

> You pick a **runtime**, usually **Tokio**: an event loop plus a pool of threads. Async tasks become concurrent *and* parallel.

main.rs · Tokio starts the runtime for you

```rust
#[tokio::main]
async fn main() {
    let s = tokio::fs::read_to_string("a.txt")
        .await
        .unwrap();
    println!("{}", s);
}
```

BEYOND JAVASCRIPT · CODE

Rust in parallel: std::thread

```rust
use std::thread;

fn main() {
    let handles: Vec<_> = (1..=3)
        .map(|i| thread::spawn(move || {
            let _sum: u64 = (0..1_000_000_000u64).sum();
            println!("thread {i} done");
        }))
        .collect();

    for h in handles { h.join().unwrap(); }
}
```

BEYOND JAVASCRIPT · CODE

Rust in parallel: std::thread

```rust
use std::thread;

fn main() {
    let handles: Vec<_> = (1..=3)
        .map(|i| thread::spawn(move || {
            let _sum: u64 = (0..1_000_000_000u64).sum();
            println!("thread {i} done");
        }))
        .collect();

    for h in handles { h.join().unwrap(); }
}
```

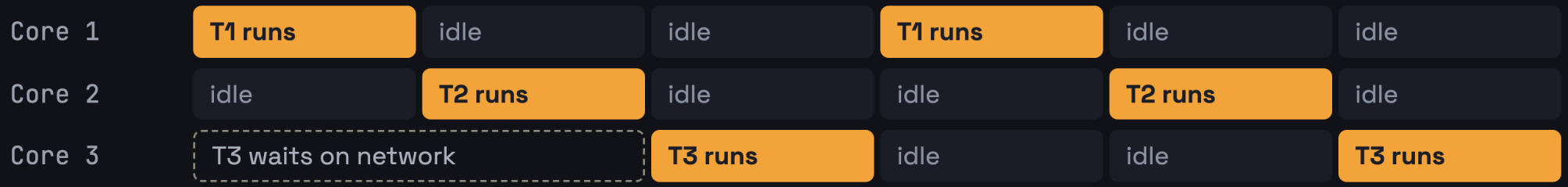
```
$ cargo run
thread 3 done
thread 1 done
thread 2 done
```

All three run at once. Try to share a plain variable between them and **it won't compile**: you need **Arc<Mutex<T>>**.

BEYOND JAVASCRIPT

What the cores are doing

Python: 3 threads, 3 cores, the GIL



Read it column by column: **never two amber blocks at the same moment.** Waiting on I/O doesn't need the GIL, so it can overlap.

Rust: 3 threads, 3 cores, no GIL

BEYOND JAVASCRIPT

Side by side

	JavaScript (Node)	Python	Rust
Runs your code on	One main thread	Many threads, one at a time (GIL)	Many threads, truly parallel

BEYOND JAVASCRIPT

Side by side

	JavaScript (Node)	Python	Rust
Runs your code on	One main thread	Many threads, one at a time (GIL)	Many threads, truly parallel
Async model	Built-in event loop, always on	asyncio event loop you start	Futures + a runtime (Tokio)

BEYOND JAVASCRIPT

Side by side

	JavaScript (Node)	Python	Rust
Runs your code on	One main thread	Many threads, one at a time (GIL)	Many threads, truly parallel
Async model	Built-in event loop, always on	asyncio event loop you start	Futures + a runtime (Tokio)
Heavy CPU work in parallel	worker_threads	multiprocessing	Plain threads

BEYOND JAVASCRIPT

Side by side

	JavaScript (Node)	Python	Rust
Runs your code on	One main thread	Many threads, one at a time (GIL)	Many threads, truly parallel
Async model	Built-in event loop, always on	asyncio event loop you start	Futures + a runtime (Tokio)
Heavy CPU work in parallel	worker_threads	multiprocessing	Plain threads
Threads share memory?	No, workers pass messages	Yes, guarded by the GIL	Yes, checked by the compiler

Same goal everywhere: **never let the worker stand around waiting.** Each language just picks different tools.

RECAP

One idea, three syntaxes

Callbacks

```
readFile(f, (err, d) ⇒ …)
```

Hand over a function to call later

RECAP

One idea, three syntaxes

Callbacks

```
readFile(f, (err, d) ⇒ …)
```

Hand over a function to call later

Promises

```
readFile(f).then(d ⇒ …)
```

Get a buzzer, attach handlers

RECAP

One idea, three syntaxes

Callbacks

```
readFile(f, (err, d) ⇒ …)
```

Hand over a function to call later

Promises

```
readFile(f).then(d ⇒ …)
```

Get a buzzer, attach handlers

async / await

```
const d = await readFile(f)
```

Pause the function, not the program

JavaScript has **one worker.** Async keeps it from ever **standing around waiting.**

Questions?

Assignment 01 is due **Thursday, 11 AM IST.**

`nodejs.org/api/fs.html · MDN: “Using promises”`